

MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: Applications of Linked Lists 1) List Based Priority Queues (Deletion) 2) Double Ended Queues
Roll No. : MCA2511	Date : 28-11-2025

1) List Based Priority Queues (Deletion).

Code:

```
import java.util.Scanner;

class PQNode
{
    int data;
    int priority;
    PQNode next;

    public PQNode(int d, int p)
    {
        data = d;
        priority = p;
        next = null;
    } //Default Constructor
} //end of PQNode

class PQueue
{
    PQNode front;
    PQNode rear;

    public PQueue()
    {
        front = null;
        rear = null;
    } //Default Constructor

    //Enqueue in order of Priority
    public void Enqueue(int d, int p)
    {
        //Make a new Node
        PQNode t = new PQNode(d,p);

        //First Node in th PQ
        if(front == null)
        {
            front = t;
            rear = t;
            return;
        }
    }
}
```

```
}

//Traverse based on Priority

PQNode tmp = front;
PQNode prev = null;

while(tmp != null && tmp.priority < t.priority)
{
    prev = tmp;
    tmp = tmp.next;
}

//Insert node based on priority
if(tmp == front)                //Front NNode Insertion
{
    t.next = front;
    front = t;
}
else if(tmp == null)
{
    rear.next = t;
    rear = t;
}
else
{
    prev.next = t;
    t.next = tmp;
}
}

//end of Enqueue

//Dequeue
public void Dequeue()
{
    //Check Empty Queue
    if(front == null)
    {
        System.out.println("Queue Underflowed...");
        return;
    }

    System.out.println("Element Removed is " + front.data + " with priority: " +
front.priority);
    if(front==rear)                //Single element Deletion
    {
        front= null;
        rear = null;
    }
    else
    {
        front = front.next;
    }
}
```

```
        }
        System.out.println();
    } //end of Dequeue

    //Peek Front
    public void PeekFront()
    {
        PQNode tmp = front;

        //Check Empty Queue
        if(front == null)
        {
            System.out.println("Queue Underflowed...");
            return;
        }

        System.out.println("\nPriority Front contains---");
        System.out.println("Elements\tPriority- ");
        System.out.println(tmp.data + "\t\t" + tmp.priority);
        System.out.println();
    } //end of PeekFront

    //Peek Rear
    public void PeekRear()
    {
        PQNode tmp = front;

        //Check Empty Queue
        if(front == null)
        {
            System.out.println("Queue Underflowed...");
            return;
        }

        System.out.println("\nPriority Rear contains---");
        System.out.println("Elements\tPriority- ");
        while (tmp != null)
        {
            if(tmp.next == null)
            {
                System.out.println(tmp.data + "\t\t" + tmp.priority);
            }
            tmp = tmp.next;
        }
        System.out.println();
    } //end of PeekRear

    //Display
    public void Display()
    {
        PQNode tmp = front;
```

```
//Check Empty Queue
if(front == null)
{
    System.out.println("Queue Underflowed...");
    return;
}

System.out.println("\nPriority Queue contains---");
System.out.println("Elements\tPriority- ");
while (tmp != null)
{
    System.out.println(tmp.data + "\t\t" + tmp.priority);
    tmp = tmp.next;
}
System.out.println();
} //end of Display
} //end of PQueue

class PQ
{
    public static void main(String[] args)
    {
        PQueue p = new PQueue();
        Scanner sc = new Scanner(System.in);
        int ch,x,y;

        do
        {
            System.out.println("\n-----");
            System.out.println("-----Priority Queue-----");
            System.out.println("-----");
            System.out.println("1. Enqueue in Priority Queue");
            System.out.println("2. Dequeue from Priority Queue");
            System.out.println("3. Peek Front in Priority Queue");
            System.out.println("4. Peek Rear in Priority Queue");
            System.out.println("5. Display Priority Queue");
            System.out.println("6. Exit.....\n");

            System.out.print("Enter your choice: ");
            ch = sc.nextInt();

            switch(ch)
            {
                case 1:
                    System.out.print("Enter your value: ");
                    x = sc.nextInt();
                    System.out.print("Enter your Priority: ");
                    y = sc.nextInt();
                    p.Enqueue(x,y);
                    break;
```

```
        case 2:
            p.Dequeue();
            break;

        case 3:
            p.PeekFront();
            break;

        case 4:
            p.PeekRear();
            break;

        case 5:
            p.Display();
            break;

        case 6:
            System.out.println("Exiting ... :");
            break;

        default:
            System.out.println("Incorrect choice :(");
            break;
    }
} while (ch != 6);

    } //end of psvm
} //end of PQ
```

Output:

```
A:\MCA2511\DS_LAB>javac 171PQ.java
```

```
A:\MCA2511\DS_LAB>java PQ
```

```
-----  
-----Priority Queue-----  
-----
```

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

```
Enter your choice: 1  
Enter your value: 12  
Enter your Priority: 4
```

```
-----  
-----Priority Queue-----  
-----
```

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

```
Enter your choice: 1  
Enter your value: 56  
Enter your Priority: 1
```

```
-----  
-----Priority Queue-----  
-----
```

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

```
Enter your choice: 1  
Enter your value: 56  
Enter your Priority: 6
```

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 5

Priority Queue contains---

Elements	Priority-
56	1
12	4
56	6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 1
Enter your value: 23
Enter your Priority: 5

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 5

Priority Queue contains---

Elements	Priority-
56	1
12	4
23	5
56	6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 3

Priority Front contains---
Elements Priority-
56 1

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 4

Priority Rear contains---
Elements Priority-
56 6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 2

Element Removed is 56 with priority: 1

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 5

Priority Queue contains---

Elements	Priority-
12	4
23	5
56	6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 2

Element Removed is 12 with priority: 4

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 2

Element Removed is 23 with priority: 5

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 2

Element Removed is 56 with priority: 6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 2

Queue Underflowed...

2) Double Ended Queue.

Code:

```
import java.util.Scanner;

//Node template
class Node
{
    int data;
    Node prev;
    Node next;

    public Node(int d)
    {
        data = d;
        prev = null;
        next = null;
    }
} //end of Node class

//Double Ended Queue Template
class DQueue
{
    Node front;
    Node rear;

    public DQueue()
    {
        front = null;
        rear = null;
    } //end of DQueue

    //Enqueue Front
    public void EnqueueFront(int x)
    {
        //Make a new node
        Node t = new Node(x);

        //First Node
        if(front==null)
        {
            front = t;
            rear = t;
        }
        else //Attach t to front
        {
            t.next = front;
            front.prev = t;
            front = t;
        }
    } //end of EnqueueFront
}
```

```
//Enqueue Rear
public void EnqueueRear(int x)
{
    //Make a new node
    Node t = new Node(x);

    //First Node
    if(front==null)
    {
        front = t;
        rear = t;
    }
    else //Attach t to rear
    {
        t.prev = rear;
        rear.next = t;
        rear= t;
    }
} //end of EnqueueRear

//Dequeue Front
public void DequeueFront()
{
    if(front==null)
    {
        System.out.println("Queue Underflowed");
        return;
    }
    System.out.println("Element removed is " + front.data);
    if(front==rear)
    {
        front = null;
        rear = null;
    }
    else
    {
        front = front.next;
        front.prev = null;
    }
} //end of DequeueFront

//Dequeue Rear
public void DequeueRear()
{
    if(front==null)
    {
        System.out.println("Queue Underflowed");
        return;
    }
    System.out.println("Element removed is " + rear.data);
    if(front==rear)
```

```
        {
            front = null;
            rear = null;
        }
        else
        {
            rear = rear.prev;
            rear.next = null;
        }
    } //end of DequeueRear

    //Display
    public void Display()
    {
        System.out.print("Element in forward Direction: ");
        Node tmp = front;
        while(tmp!=null)
        {
            System.out.print(tmp.data+ " ");
            tmp = tmp.next;
        }

        System.out.println();

        System.out.print("Element in reverse Direction: ");
        tmp = rear;
        while(tmp!=null)
        {
            System.out.print(tmp.data+ " ");
            tmp = tmp.prev;
        }
    } //end of Display

} //end of DQueue

// Main
class DQ
{
    public static void main(String[] args)
    {
        DQueue d = new DQueue();
        Scanner sc = new Scanner(System.in);
        int ch, x;

        do
        {
            System.out.println("\n-----");
            System.out.println("Double Ended Queue");
            System.out.println("-----\n");

            System.out.println("1. Enqueue Front");
            System.out.println("2. Enqueue Rear");
```

```
System.out.println("3. Dequeue Front");
System.out.println("4. Dequeue Rear");
System.out.println("5. Display");
System.out.println("6. Exit....\n");

System.out.print("Enter your Choice: ");
ch = sc.nextInt();

switch(ch)
{
    case 1:
        System.out.print("Enter a value: ");
        x = sc.nextInt();
        d.EnqueueFront(x);
        break;

    case 2:
        System.out.print("Enter a value: ");
        x = sc.nextInt();
        d.EnqueueRear(x);
        break;

    case 3:
        d.DequeueFront();
        break;

    case 4:
        d.DequeueRear();
        break;

    case 5:
        d.Display();
        break;

    case 6:
        System.out.println("Exiting ..... :");
        break;

    default:
        System.out.println("Incorrect Choice .... :(");
        break;
}
}while (ch != 6);
} //end of psvm
} //end of DQ
```


Output:

insert 10 20 30 40 50 using EnqueueFront

A:\MCA2511\DS_LAB>javac 18DQ.java

A:\MCA2511\DS_LAB>java DQ

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 1
Enter a value: 10

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 1
Enter a value: 20

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 1
Enter a value: 30

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 1
Enter a value: 40

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 1
Enter a value: 50

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 5
Element in forward Direction: 50 40 30 20 10
Element in reverse Direction: 10 20 30 40 50

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 2
Enter a value: 60

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 2
Enter a value: 70

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 2
Enter a value: 80

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 5
Element in forward Direction: 50 40 30 20 10 60 70 80
Element in reverse Direction: 80 70 60 10 20 30 40 50

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 3
Element removed is 50

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 3
Element removed is 40

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 5
Element in forward Direction: 30 20 10 60 70 80
Element in reverse Direction: 80 70 60 10 20 30

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 4
Element removed is 70

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 4
Element removed is 60

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 5
Element in forward Direction: 30 20 10
Element in reverse Direction: 10 20 30

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 3
Element removed is 30

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 4
Element removed is 10

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 3
Element removed is 20

Double Ended Queue

1. Enqueue Front
2. Enqueue Rear
3. Dequeue Front
4. Dequeue Rear
5. Display
6. Exit....

Enter your Choice: 3
Queue Underflowed